



UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

**Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ciencia de la Computación**

Proyecto Final

Docente: Percy Maldonado Quispe

Alumnos:

Prince Yorwin Mariños Hilario

Gleddynuri Marbel Picha Chañi

Wilson Ramos Pacco

Postigo Cabana Juan Carlos

Curso: Computación Gráfica
Arequipa — Perú
2026

Índice

1. Introducción y alcance	2
2. Organización del proyecto	2
3. Arquitectura del sistema	2
3.1. WPoint — el vértice	2
3.2. Wave — la ola	3
3.3. Ocean — el controlador	3
4. Generación de la malla base	3
5. Animación del oleaje	4
6. Iluminación	5
6.1. Por qué hace falta una normal en cada vértice	5
6.2. El modelo de iluminación: Phong / Blinn-Phong	6
6.3. Configuración real en el proyecto: luz y material	6
6.4. Evidencia visual: el brillo especular	8
7. Texturizado	9
8. Cámara interactiva	10
9. Estado pendiente	11
10. Conclusiones parciales	11

1. Introducción y alcance

Este documento describe el estado actual del proyecto *Sistema de Animación de Océano*: una superficie de agua animada mediante superposición de ondas sinusoidales, con normales por vértice, iluminación con material especular, texturizado real y cámara orbital interactiva. El código fuente y los recursos están disponibles en el repositorio del grupo:

<https://github.com/PrincceMH/VisualizacionOcean>

Quedan pendientes, y se detallan en la Sección 9, los controles adicionales de teclado y los elementos extra de la escena.

2. Organización del proyecto

El código se organiza separando fuentes, cabeceras y recursos:

```
VisualizacionOcean/
|
+-- src/
| +-- main.cpp
| +-- Ocean.cpp
| +-- Wave.cpp
|
+-- include/
| +-- Ocean.h
| +-- Wave.h
| +-- WPoint.h
| +-- stb_image.h (carga de imagenes de textura)
|
+-- assets/
| +-- textures/
| +-- ocean.tga
|
+-- data/
    +-- spectrum.txt (espectro de olas: amplitud, direccion, frecuencia)
```

3. Arquitectura del sistema

El sistema se organiza en tres clases con responsabilidades separadas, tal como sugiere el enunciado del laboratorio.

3.1. WPoint — el vértice

Unidad básica de la malla. Almacena posición, normal (usada por la iluminación) y coordenadas de textura UV.

```
struct WPoint {
    float x, y, z; // Posicion en el espacio
    float nx, ny, nz; // Vector normal
    float s, t; // Coordenadas de textura UV
};
```

3.2. Wave — la ola

Encapsula los parámetros físicos de una ola individual: amplitud, frecuencia, dirección y fase. Provee el cálculo del número de onda k , necesario para la fórmula de desplazamiento (ondas de aguas profundas):

$$k = \frac{4\pi^2 f^2}{g}, \quad g = 9,81 \text{ m/s}^2$$

```
class Wave {
    float amplitude, frequency, direction, phase;
public:
    Wave(float amp, float freq, float dir, float ph);
    float getWaveNumber() const; // Calcula k
};
```

3.3. Ocean — el controlador

Administra la malla 2D de vértices, la lista de olas, el identificador de la textura de OpenGL y las operaciones de ciclo de vida: inicialización, actualización de altura, cálculo de normales, carga de textura y dibujo.

```
class Ocean {
    std::vector<std::vector<WPoint>>> mesh;
    std::vector<Wave> waves;
    unsigned int textureID;
public:
    void initMesh();
    bool loadWaves(const std::string& filename);
    bool loadTexture(const std::string& filename);
    void update(float time);
    void computeNormals();
    void draw();
};
```

4. Generación de la malla base

La malla se construye sobre el plano XZ , centrada dinámicamente respecto al origen. Cada vértice guarda también su coordenada de textura (s, t) , proporcional a su posición en la rejilla (0 a 1), lista para usarse en el texturizado:

```
void Ocean::initMesh() {
    mesh.resize(rows, std::vector<WPoint>(cols));

    float startX = -((cols - 1) * spacing) / 2.0f;
    float startZ = -((rows - 1) * spacing) / 2.0f;

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            mesh[i][j].x = startX + j * spacing;
            mesh[i][j].y = 0.0f;
            mesh[i][j].z = startZ + i * spacing;
```

```

        mesh[i][j].s = (float)j / (cols - 1);
        mesh[i][j].t = (float)i / (rows - 1);
    }
}

```

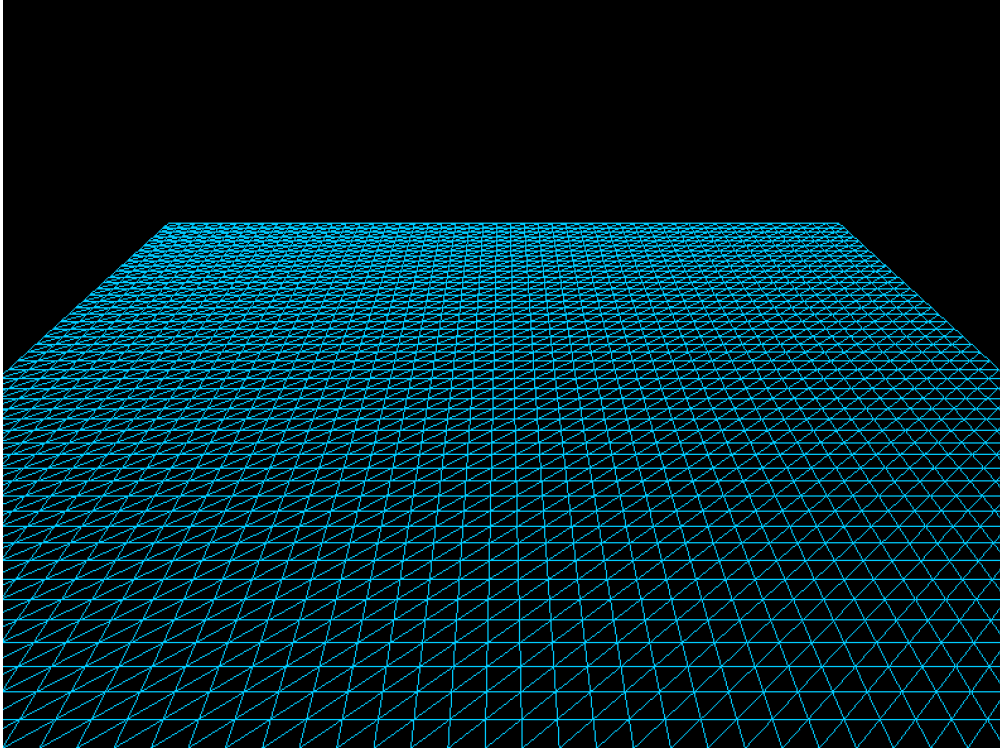


Figura 1: Malla base plana renderizada en modo alambre, usada para validar la topología antes de animarla.

5. Animación del oleaje

La altura de cada vértice se recalcula cada cuadro sumando la contribución de todas las olas del espectro:

$$h(x, z, t) = \sum_{i=1}^n A_i \cos(k_i(x \cos(d_i) + z \sin(d_i)) - 2\pi f_i t + p_i)$$

`Ocean::loadWaves()` lee amplitud, dirección y frecuencia de cada ola desde `data/spectrum.txt` (una por línea) y genera una fase aleatoria para cada una. Si el archivo no se encuentra, se usan olas por defecto para que la simulación no quede plana:

```

bool Ocean::loadWaves(const std::string& filename) {
    // busca el archivo en varias rutas relativas posibles...
    if (!file.is_open()) {
        waves.push_back(Wave(0.5f, 0.15f, 30.0f * PI/180.0f, 0.0f));
        waves.push_back(Wave(0.3f, 0.25f, 90.0f * PI/180.0f, 60.0f * PI/180.0f));
        waves.push_back(Wave(0.2f, 0.40f, 150.0f * PI/180.0f, 120.0f * PI/180.0f));
    }
}

```

```

    return false;
}
// ... si abre el archivo, carga amplitud/direccion/frecuencia por linea
// y genera una fase aleatoria para cada ola
}

```

`Ocean::update(float time)` recorre la malla y evalúa $h(x, z, t)$; al final llama a `computeNormals()` para que las normales queden al día con la nueva forma de la superficie:

```

void Ocean::update(float time) {
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j) {
            float height = 0.0f;
            for (const Wave& wave : waves) {
                float k = wave.getWaveNumber();
                float theta = k * (x*cosf(wave.getDirection()) + z*sinf(wave.getDirection())
                    - 2.0f * PI * wave.getFrequency() * time + wave.getPhase());
                height += wave.getAmplitude() * cosf(theta);
            }
            mesh[i][j].y = height;
        }
    computeNormals();
}

```

En `main.cpp`, un `glutTimerFunc` avanza el tiempo de simulación y solicita el redibujado en cada cuadro (~60 veces por segundo), logrando la animación continua.

6. Iluminación

Esta sección reúne, de forma justificada, todo lo que hace falta para que el océano se vea iluminado de manera realista: primero *por qué* se necesita una normal en cada vértice, luego el modelo matemático que OpenGL usa para combinar luz y material, y finalmente el código real del proyecto que configura ambos. El orden importa: cada pieza depende conceptualmente de la anterior.

6.1. Por qué hace falta una normal en cada vértice

La iluminación de OpenGL (función fija) no “sabe” que la superficie es un océano ondulado: solo sabe, para cada vértice, un vector llamado **normal** que indica hacia dónde mira ese punto de la superficie. Sin ese dato, es imposible calcular si el vértice está de frente a la luz o de canto, y por lo tanto tampoco cuánta luz recibe.

En una malla *plana* la normal sería siempre $(0, 1, 0)$ para todos los puntos. Pero aquí la altura de cada vértice cambia constantemente (Sección 5), así que la superficie está inclinada de forma distinta en cada punto y en cada instante; la normal debe **recalcularse cada cuadro**, junto con la altura.

`Ocean::computeNormals()` calcula la normal de cada triángulo de la malla con el **producto vectorial** de dos de sus aristas ($\vec{n}_{\Delta} = \vec{e}_1 \times \vec{e}_2$), y la **acumula** en sus tres vértices; como cada vértice es compartido por varios triángulos vecinos, al final se **normaliza el promedio** acumulado. Esta es exactamente la técnica que recomienda el enunciado del proyecto:

```

void Ocean::computeNormals() {

```

```

// 1) reiniciar el acumulador de cada vertice a cero
// 2) por cada triangulo de la malla:
// normal_de_cara = (arista1) x (arista2) [producto cruz]
// se suma esa normal a sus 3 vertices
// 3) cada vertice normaliza su acumulado (el promedio de sus caras vecinas)
}

```

Por qué promediar y no dejar la normal de una sola cara. Si cada triángulo usara únicamente su propia normal, la transición entre triángulos vecinos sería abrupta y la superficie se vería facetada, como un cristal tallado en vez de agua. Promediar las normales de las caras que rodean a cada vértice suaviza esa transición: el resultado es un degradado continuo de luz sobre la superficie, aunque por debajo la malla siga siendo un conjunto de triángulos planos.

6.2. El modelo de iluminación: Phong / Blinn-Phong

Con la normal ya disponible, OpenGL calcula el color final de cada vértice combinando tres contribuciones de luz –ambiental, difusa y especular– cada una resultado de multiplicar la componente de la **luz** por la componente equivalente del **material**:

$$I_{\text{total}} = \underbrace{k_a L_a}_{\text{ambiental}} + \underbrace{k_d L_d (\hat{N} \cdot \hat{L})}_{\text{difusa}} + \underbrace{k_s L_s (\hat{N} \cdot \hat{H})^{\text{shininess}}}_{\text{especular}}$$

donde $\hat{N}$ es la normal del vértice, $\hat{L}$ es la dirección hacia la luz, y $\hat{H}$ es el **vector medio** entre la dirección hacia la luz y la dirección hacia la cámara, $\hat{H} = \frac{\hat{L} + \hat{V}}{2}$ (el modelo de iluminación de función fija de OpenGL usa esta variante, llamada *Blinn-Phong*, en lugar del vector de reflexión especular puro; es matemáticamente más barata y visualmente casi idéntica).

Ambiental ($k_a L_a$): no depende de ningún ángulo; es una luz de relleno constante, la razón por la que los valles de las olas nunca quedan completamente negros.

Difusa ($k_d L_d (\hat{N} \cdot \hat{L})$): depende del **ángulo entre la normal y la luz**. El producto punto $\hat{N} \cdot \hat{L}$ es máximo (=1) cuando la superficie mira directo a la luz, y decrece a medida que se inclina; esto es lo que da **volumen** a la superficie ondulada.

Especular ($k_s L_s (\hat{N} \cdot \hat{H})^{\text{shininess}}$): el brillo puntual y concentrado. Es máximo solo cuando la normal apunta casi exactamente a mitad de camino entre la luz y la cámara; el exponente **shininess** controla qué tan concentrado (vs. disperso) se ve ese reflejo.

6.3. Configuración real en el proyecto: luz y material

`initLuzYMaterial()`, en `main.cpp`, activa el sistema de iluminación de OpenGL y define las tres componentes (L_a, L_d, L_s) de la luz y las tres componentes (k_a, k_d, k_s) del material del agua, además del exponente de brillo:

```

void initLuzYMaterial() {
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_NORMALIZE);

    GLfloat luzAmbiental[] = { 0.25f, 0.25f, 0.28f, 1.0f };
}

```

```

GLfloat luzDifusa[] = { 0.85f, 0.85f, 0.80f, 1.0f };
GLfloat luzEspecular[] = { 0.50f, 0.50f, 0.52f, 1.0f };
glLightfv(GL_LIGHT0, GL_AMBIENT, luzAmbienta);
glLightfv(GL_LIGHT0, GL_DIFFUSE, luzDifusa);
glLightfv(GL_LIGHT0, GL_SPECULAR, luzEspecular);

// Material del oceano: azul profundo, con brillo especular moderado
GLfloat waterAmbient[] = { 0.05f, 0.10f, 0.15f, 1.0f };
GLfloat waterDiffuse[] = { 0.10f, 0.35f, 0.55f, 1.0f };
GLfloat waterSpecular[] = { 0.45f, 0.52f, 0.58f, 1.0f };
glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, waterAmbient);
glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, waterDiffuse);
glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, waterSpecular);
glMaterialf (GL_FRONT_AND_BACK, GL_SHININESS, 48.0f);
}

```

`GL_NORMALIZE` es una salvaguarda necesaria por la forma en que se calculan las normales: como `computeNormals()` **acumula** varias normales antes de dividir por su promedio, un vector intermedio podría no tener longitud exactamente 1 al llegar a la etapa de iluminación. Con `GL_NORMALIZE` activo, OpenGL renormaliza cualquier normal antes de usarla en la fórmula de Phong; sin esto, el resultado de $\hat{N} \cdot \hat{L}$ quedaría mal escalado y la superficie se vería más oscura o más clara de lo que corresponde.

La posición de la luz se actualiza en `display()` *después* de `gluLookAt`:

```

gluLookAt(ex, ey, ez, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);

// Actualiza posicion de luz despues de gluLookAt (anclada al mundo)
GLfloat posicionLuz[] = { 10.0f, 15.0f, 10.0f, 1.0f };
glLightfv(GL_LIGHT0, GL_POSITION, posicionLuz);

```

Por qué en ese orden. La posición de una luz se ve afectada por la transformación de vista vigente en el momento en que se define (igual que cualquier otro punto que se pasa a OpenGL). Fijarla *después* de `gluLookAt` la deja anclada al **mundo**: un sol fijo que no se mueve al orbitar la cámara. Fijarla antes la dejaría pegada a la cámara, como una linterna en la cabeza del observador.

6.4. Evidencia visual: el brillo especular

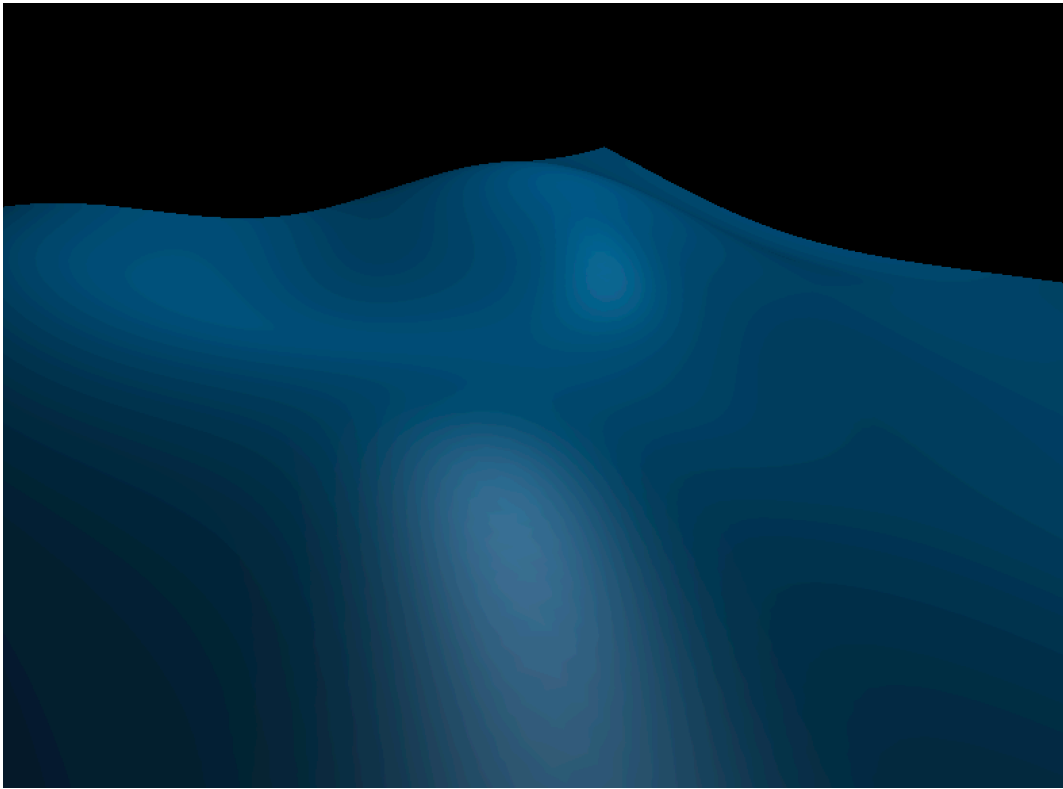


Figura 2: Océano con geometría e iluminación, *sin* textura (capturado desactivando temporalmente `glEnable(GL_TEXTURE_2D)` en `Ocean::draw()`). Se distinguen dos manchas de brillo especular: una grande en el centro y otra más pequeña arriba a la derecha, cada una sobre una cresta distinta de la superficie.

Por qué el brillo aparece en esos dos puntos y no en toda la superficie. Según la fórmula especular de la sección anterior, el reflejo solo se enciende con fuerza donde $\hat{N} \cdot \hat{H} \approx 1$, es decir, donde la normal local coincide con el vector medio entre la luz y la cámara. Como la normal de cada vértice apunta en una dirección distinta según la inclinación local de la ola, solo las crestas orientadas “justo hacia” esa combinación luz-cámara producen el reflejo brillante; el resto de la superficie recibe la misma luz pero en un ángulo que aleja $\hat{H}$ de $\hat{N}$, y por eso se ve con el tono difuso azul, más apagado. Este efecto es completamente dependiente de la normal calculada en la Sección 6.1: sin normales correctas (o sin normalizar) el brillo aparecería disperso o en el lugar equivocado.

Por qué hay dos manchas y no una sola. Con una única ola habría un solo punto de brillo (como en una esfera lisa). Aquí el océano es la **suma de varias ondas sinusoidales** con direcciones distintas (Sección 5), así que existen varias crestas locales con orientaciones distintas; cada una que “mira” hacia la combinación correcta de luz y cámara genera su propio punto de brillo independiente. El número y la posición de estos brillos cambian cuadro a cuadro porque las crestas se mueven con la animación del oleaje.

7. Texturizado

La superficie se texturiza con una imagen real de agua (`assets/textures/ocean.tga`), cargada con la librería de una sola cabecera `stb_image.h`:

```
bool Ocean::loadTexture(const std::string& filename) {
    glGenTextures(1, &textureID);
    glBindTexture(GL_TEXTURE_2D, textureID);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    stbi_set_flip_vertically_on_load(true);
    unsigned char* data = stbi_load(filename.c_str(), &width, &height, &nrChannels, 0);
    GLenum format = (nrChannels == 4) ? GL_RGBA : GL_RGB;
    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE,
        data);
    stbi_image_free(data);
}
```

Al dibujar, cada vértice emite su coordenada de textura junto con su normal y posición (`glTexCoord2f` antes de `glVertex3f`), y `GL_MODULATE` combina el color de la textura con el resultado de la iluminación, de forma que el agua se ve iluminada *y* con su patrón visual a la vez.

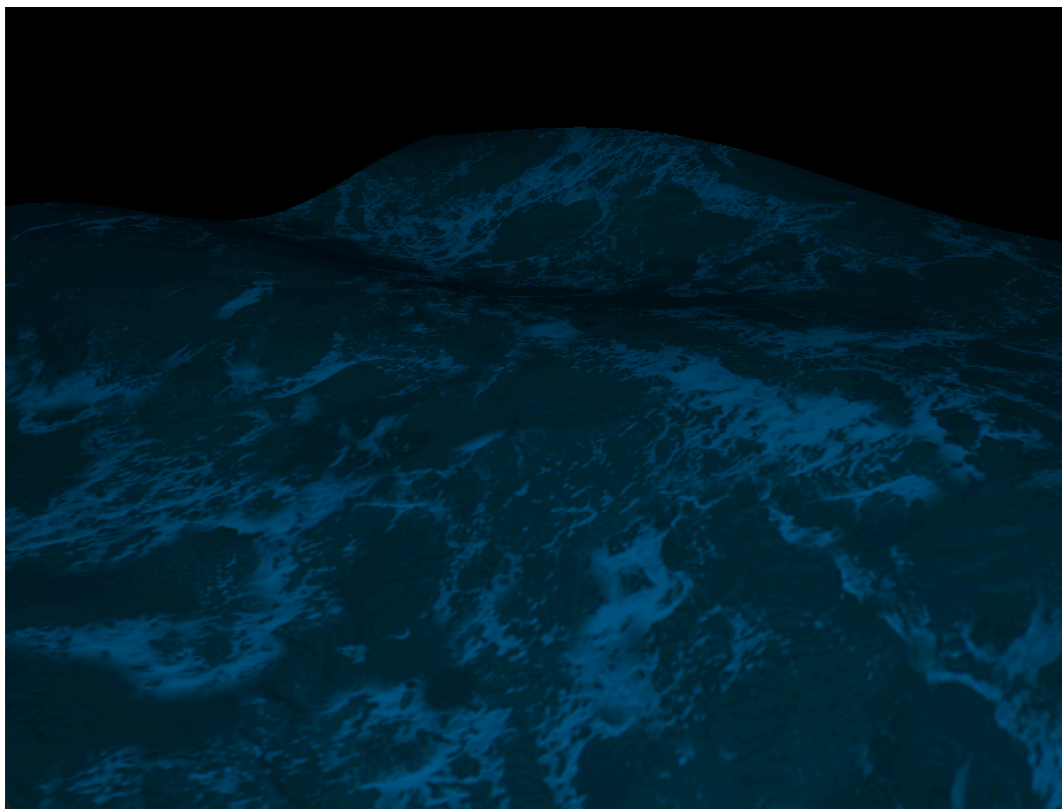


Figura 3: Océano con textura, normales e iluminación combinadas. La forma general de las olas la aporta la **geometría** (la malla deformada por `update()`); el detalle fino de espuma y reflejos lo aporta la **textura** (`ocean.tga`); y el contraste entre la cresta iluminada y los valles en sombra lo aporta la **iluminación**, que usa las normales calculadas por `computeNormals()` para saber cuánta luz recibe cada punto de la superficie.

Por qué la imagen no se ve estirada. Aunque las coordenadas (s, t) de cada vértice van de 0 a 1 sobre *toda* la malla, la imagen de `ocean.tga` ya contiene en sí misma un patrón de espuma a muchas escalas (grande y fino a la vez). Por eso una sola copia, estirada sobre el océano completo, sigue leyéndose como agua real: el detalle no depende de repetir la textura en mosaico, sino de que la fotografía original ya trae variación a distintas frecuencias espaciales. Esto contrasta con una textura de patrón simple y regular (como el ladrillo del Lab 9), donde estirar una sola copia sí produce un resultado visiblemente deformado y sí hace falta repetirla (`GL_REPEAT` con coordenadas mayores a 1) para que se vea correcta.

8. Cámara interactiva

`main.cpp` implementa una cámara orbital con **coordenadas esféricas** (radio, ángulo horizontal y vertical), igual que en los laboratorios previos del curso: arrastrar el ratón cambia los ángulos, y las teclas +/- acercan o alejan la cámara. La posición resultante se pasa a `gluLookAt` en cada cuadro.

9. Estado pendiente

Lo que aún falta para cubrir por completo los requerimientos del enunciado:

- **Controles de teclado en tiempo real:** aumentar/disminuir la velocidad de la animación, y alternar textura e iluminación on/off.
- **Elementos adicionales de la escena** (mínimo dos, integrados con posición, escala, iluminación y materiales propios): por ejemplo un cielo/fondo, un barco flotando sobre las olas, o el sol.
- **Entrega final:** video corto de la animación.

10. Conclusiones parciales

El proyecto cuenta con una arquitectura orientada a objetos clara (Wave, WPoint, Ocean), la malla se anima con superposición de ondas sinusoidales leídas desde un archivo de espectro, las normales se calculan promediando las caras vecinas de cada vértice tal como recomienda el enunciado, y la escena ya integra iluminación con material especular, una textura real de agua y una cámara orbital interactiva. El trabajo restante es acotado: controles de teclado adicionales y los elementos extra de la escena.